Exercise 2 — Build the pipeline, then break it on purpose

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Set 2 October 2026 · discussed at the start of lesson 3, Friday 9 October 2026

Contents

Roughly 2–3 hours. Work alone; discussing ideas with others is fine, sharing notebooks is not.

Goal

Build a complete, leak-free preparation pipeline for a messy dataset, then — this is the part that carries the most marks — **deliberately construct a leak, measure what it manufactures, and fix it.** Anyone can follow the rule "scale inside the pipeline" by rote. This exercise checks that you understand *why*, by having you break it on purpose and see the number it produces.

Dataset

The same synthetic telecom churn dataset used in the lesson, from a different random seed so your numbers will not match anyone else's. Copy `churn_data.py` — from this lesson's `Notebooks/` folder — into the same folder as your notebook, then:

```
from churn_data import make_churn_data
```

```
df = make_churn_data(n=2000, seed=104)
```

Keep the seed fixed at `104` so your results are reproducible and your markdown commentary matches the numbers your code actually produces.

The columns are the same as in the lesson: numeric (`tenure_months`, `monthly_charges`, `age`, `num_support_calls`), low-cardinality categorical (`contract_type`, `region`), a high-cardinality categorical (`zip_code`), and the target `churned`. Nothing about their meaning has changed — only the random draw.

Tasks

1. Look, before anything else

Report the shape, the dtypes, the churn rate and the resulting baseline accuracy. Report which columns have missing values and by how much. For each, state whether you believe it is missing completely at random (MCAR) or missing at random (MAR), and what observation led you to that judgement (you do not have access to the generating code's internals — argue from what you can observe, the way you would with a real dataset).

2. Split, before anything else

Hold out a test set. State your proportion, and whether you **stratified** — kept each class in the proportion it has in the whole dataset — and why.

3. Outliers

Using both the z-score and interquartile range (IQR) rules from the lesson, find candidate outliers in `tenure_months` and `monthly_charges`. Then apply **at least one domain rule** that neither statistical rule would catch on its own, and explain what it catches that the statistical rules miss.

4. Missing values

Choose and justify a treatment for each column with missing values. If you use an indicator column for a MAR column, say why. Whatever you choose, it must be **fitted on the training fold only** — say in one sentence where in your code that is enforced.

5. The honest pipeline

Build a single `ColumnTransformer` + `Pipeline` that imputes, scales the numeric columns, encodes the low-cardinality categoricals, and fits a `LogisticRegression`. Do **not** include `zip_code` yet. Report baseline accuracy, model accuracy, and model area under the receiver operating characteristic curve (AUC) on the test set, and explain in one or two sentences why AUC is the more informative number here.

6. Build a leak on purpose

Pick **one** of the following and implement it exactly as described — the "wrong" way, deliberately:

- **(a) Leaky imputation.** Standardise and then impute the numeric columns with `KNNImputer` — in that order, since it measures distance — with **both** steps fitted on the concatenation of your training and test data, *before* using the result to train and evaluate a model.

  Be warned about this branch before you choose it: the gap it produces is smaller than the difference between two random splits, and it comes out negative about half the time. That is not you doing it wrong. It is the finding, and Task 7 asks you to establish it rather than explain it away.

- **(b) Leaky encoding.** Encode `zip_code` by the mean churn rate of its members, computed over the **whole** dataset (train and test together), and add it as a feature before training and evaluating a model.

Report the resulting test AUC. Then build the honest version of the same step (imputation fitted on the training fold only inside the pipeline; or `sklearn.preprocessing.TargetEncoder` fitted inside the pipeline for the encoding option) and report its test AUC alongside the leaky one.

7. Explain the gap — or establish that there isn't one

If you chose (b), the encoding leak. Explain *why* the two numbers differ, using the mechanism from handout Section 9.2 rather than restating "it leaked." Then find at least one specific `zip_code` value in your data for which the leak is especially large or especially small, and explain why, using the category-size argument.

If you chose (a), the imputation leak. Your two numbers will differ by very little, and possibly in the wrong direction. Do not explain that away — measure it. Redraw the split with at least ten seeds, report the distribution of (leaky − honest), and say whether the leak is distinguishable from split noise.

Then answer the question that matters: **the leak certainly happened — what is your evidence?** The AUC is not it. Produce the evidence the way notebook 3 does, by finding the training rows whose missing value was filled using a donor from the test set, and report how many of them there are. Finally, say why the metric stays silent here, using what your own Task 1 and the correlation with `churned` tell you about how much `age` matters.

This branch is worth full marks. A leak you can prove happened and prove did not move your score is a more useful thing to have understood than one that announces itself.

8. Conclude

In no more than 150 words: which of your preprocessing decisions were you least sure about, and what would you check first if you had another day with this dataset?

What to produce, and keep

A single notebook named `exercise02.ipynb` that:

- runs top to bottom in the course container with no edits beyond the file path to `churn_data.py`
- has `random_state` fixed everywhere, so your numbers reproduce
- keeps its outputs, so it can be read without being run

- contains **both** the leaky and the honest version of your chosen Task 6 step, side by side, with their scores clearly labelled

---

Assessment criteria

Criterion	Weight	What earns marks
Methodological correctness	40%	Split before anything is learned; every fitted step lives inside the pipeline in the honest version; the domain-rule outlier check is genuinely independent of the statistical ones
Implementation	30%	The leak is implemented *exactly* as specified (not a weaker version of it); both the leaky and honest numbers are reported; code is clean and reproducible
Interpretation and communication	30%	Task 7 uses the actual mechanism, not just the word "leakage" — and, on option (a), distinguishes "the leak happened" from "the leak moved my score" instead of conflating them; the missingness-mechanism judgement in Task 1 is argued from evidence; Task 8 is honest about uncertainty

There are **no marks for how large or small the leak turns out to be** — a small, correctly explained gap earns full marks; a large, unexplained one does not. As in Exercise 1, a notebook that leaves a fitted step outside the pipeline in the *honest* version loses methodology marks regardless of its scores.

Hints

- `df.groupby("zip_code")["churned"].mean()` is the one-line version of the leaky encoding in Task 6(b) — resist the urge to make it more elaborate than the handout's description; the exercise is about seeing the ordinary, unremarkable version of the bug.
- For the honest version of 6(b), `sklearn.preprocessing.TargetEncoder` inside your `ColumnTransformer` handles the cross-fitting for you; you do not need to implement leave-one-out by hand, though you are welcome to.

- Task 7's "specific `zip_code` value" is easiest to find by sorting the value counts and looking at the least frequent codes — the notebook's singleton-code example is a template for the argument, not something to reuse verbatim on data with a different seed.
- If your chosen option's leak looks tiny or negative, do not "fix" the dataset to make it bigger, and do not switch options to get a nicer number. Report what you measured. Handout Section 9.1 is about exactly this case: a leak that is real, traceable and invisible to the metric, all three at once.
- Raising the missing fraction in `churn_data.py` will not open the gap in option (a) — it has been tried. Understanding *why* not is worth more than the experiment.